

자동차IT융합학과 20241268 지윤주
자동차인공지능 이상헌교수님
2026/06/17

HW 5

Pytorch를 이용한 CNN 구현과 MNIST
숫자 분류

1. 서론

1.1. 모델 구조

이번 실험의 모델은 조건별로 스위칭이 가능한 conv-ReLU-Maxpool 기반의 레이어와 MLP가 결합된 심층 신경망 구조이다. 독립 시드 기반으로 데이터 분할, Test set 격리를 통해 데이터 오염을 방지하고 Config 클래스를 구축해서 고정 변수와 조작 변수들을 관리하며 편하게 실험하도록 만들었다.

1.2. 실험 조건

1) 데이터셋

MNIST dataset

28×28 grayscale image

10개 클래스 (0~9)

Training set: 5,000

Validation set: 1,000

Test set: 1,000

2) 통제 변인 (기준 CNN)

- Conv2d(1, 32, kernel_size=3, stride=1, padding=1)
- ReLU
- MaxPool2d(2, 2)
- Conv2d(32, 64, kernel_size=3, stride=1, padding=1)
- ReLU
- MaxPool2d(2, 2)
- Flatten
- Linear(64×7×7, 128)
- ReLU
- Linear(128, 10)
- Loss: CrossEntropyLoss
- Optimizer: Adam
- Learning rate: 0.001
- Batch size: 100
- Epoch: 10

3) 조작 변인

- Convolution Layer 수 : 1개 / 2개
- Dropout 여부 : True / False (p = 0.5)
- Optimizer : SGD / ADM

2. 본론 - 실험 결과 및 분석

2.1. 실험 결과 표

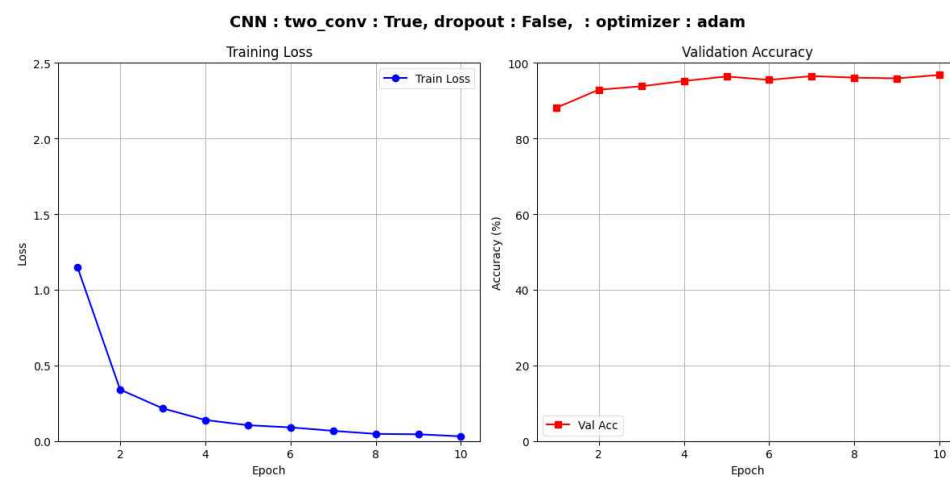
* train acc, val acc는 마지막 10th epoch의 값을 기록

* 연두색 칸은 기본 BASELINE 조건임을 표시

Experiment	Setting	Train Acc	Val Acc	Test Acc	Comment
Conv Layer	1	98.84	95.50	96.90	1분
Conv Layer	2	99.10	96.80	98.20	1분
Dropout	True	96.34	97.60	98.40	1분
Dropout	False	99.10	96.80	98.20	1분
Optimizer	SGD	21.62	21.00	21.30	1분
Optimizer	ADAM	99.10	96.80	98.20	1분

2.2. Baseline Acc 추적 (Conv Layer = 2, Dropout = False , Optimizer = ADAM)

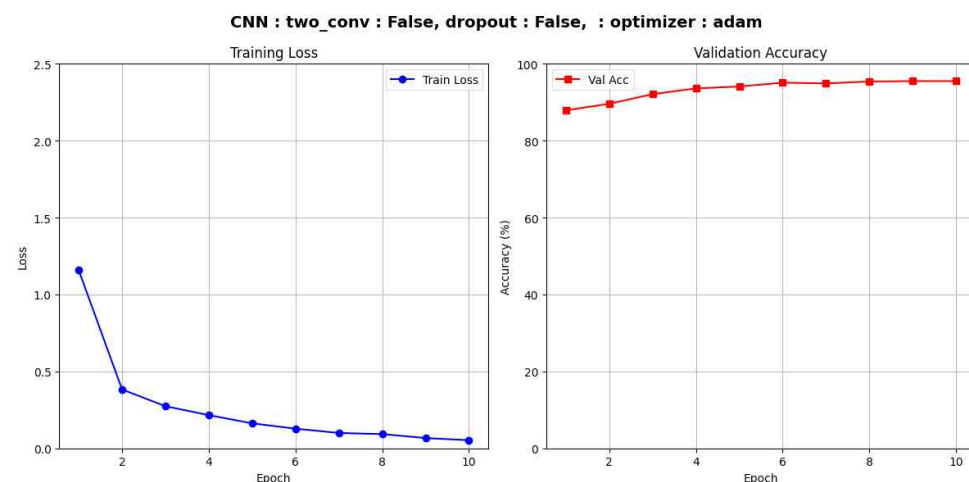
```
CNN : two_conv = True, dropout = False, optimizer = adam
Epoch 1/10 Train Loss: 1.1497 | Train Acc: 67.04% | Val Acc: 88.10%
Epoch 2/10 Train Loss: 0.3406 | Train Acc: 89.52% | Val Acc: 92.90%
Epoch 3/10 Train Loss: 0.2162 | Train Acc: 93.84% | Val Acc: 93.80%
Epoch 4/10 Train Loss: 0.1393 | Train Acc: 95.84% | Val Acc: 95.20%
Epoch 5/10 Train Loss: 0.1048 | Train Acc: 97.16% | Val Acc: 96.40%
Epoch 6/10 Train Loss: 0.0906 | Train Acc: 97.34% | Val Acc: 95.50%
Epoch 7/10 Train Loss: 0.0676 | Train Acc: 97.92% | Val Acc: 96.50%
Epoch 8/10 Train Loss: 0.0474 | Train Acc: 98.74% | Val Acc: 96.10%
Epoch 9/10 Train Loss: 0.0448 | Train Acc: 98.56% | Val Acc: 95.90%
Epoch 10/10 Train Loss: 0.0314 | Train Acc: 99.10% | Val Acc: 96.80%
Test acc : 98.20%
```



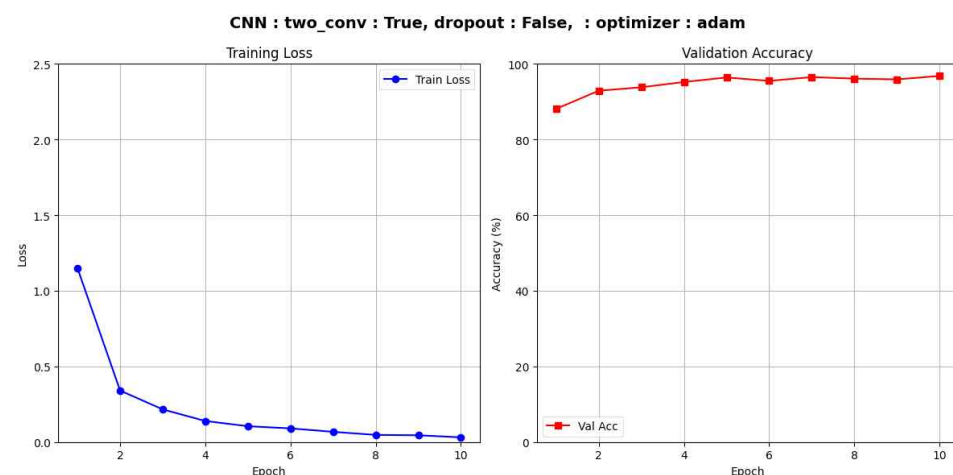
2.3. Conv Layer 비교

1) Conv Layer = 1 (Conv Layer = 1, Dropout = False, Optimizer = ADAM)

```
CNN : two_conv = False, dropout = False, optimizer = adam
Epoch 1/10 Train Loss: 1.1581 | Train Acc: 67.82% | Val Acc: 87.90%
Epoch 2/10 Train Loss: 0.3826 | Train Acc: 88.76% | Val Acc: 89.60%
Epoch 3/10 Train Loss: 0.2742 | Train Acc: 91.64% | Val Acc: 92.10%
Epoch 4/10 Train Loss: 0.2159 | Train Acc: 93.84% | Val Acc: 93.60%
Epoch 5/10 Train Loss: 0.1627 | Train Acc: 95.02% | Val Acc: 94.10%
Epoch 6/10 Train Loss: 0.1279 | Train Acc: 96.46% | Val Acc: 95.10%
Epoch 7/10 Train Loss: 0.1001 | Train Acc: 97.44% | Val Acc: 94.90%
Epoch 8/10 Train Loss: 0.0928 | Train Acc: 97.32% | Val Acc: 95.40%
Epoch 9/10 Train Loss: 0.0669 | Train Acc: 98.52% | Val Acc: 95.50%
Epoch 10/10 Train Loss: 0.0530 | Train Acc: 98.84% | Val Acc: 95.50%
Test acc : 96.90%
```



2) Conv Layer = 2 (BASELINE)

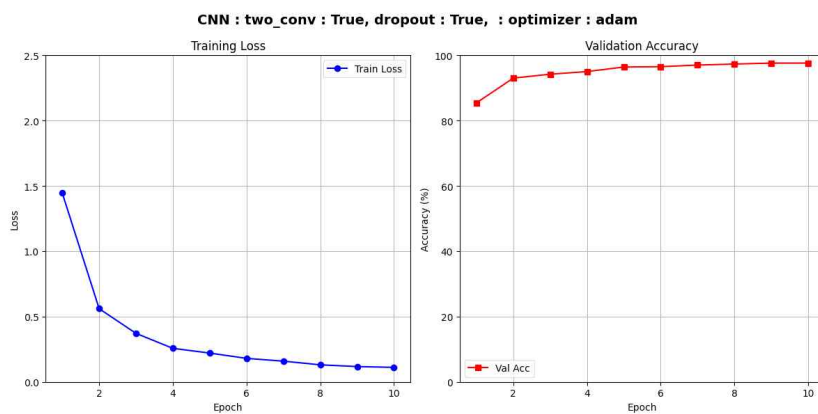


- layer가 깊어질수록 어떤 장점이 있었는가?
 - Convolution 레이어가 깊어질수록 하위 레이어에서 추출한 단순한 특징을 조합해서 상위 레이어에서 더 고차원적인 공간적 특징을 효과적으로 학습할 수 있다.
- 성능 향상이 항상 큰가?
 - MNIST와 같이 데이터셋의 해상도가 낮고 구조가 단순한 문제에서는 레이어 한 개만으로도 이미 특징 추출이 잘 되기 때문에 유의미한 성능 향상이 관찰되지 않았다.

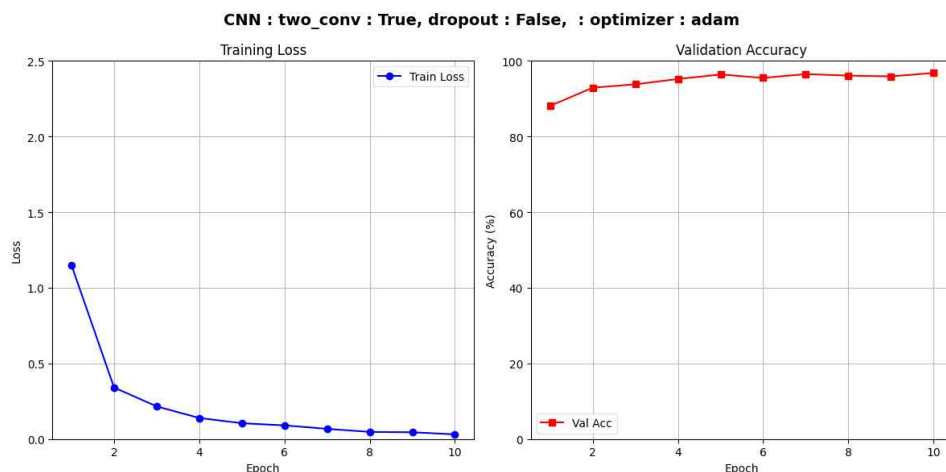
2.4. Dropout 비교

1) Dropout = True (Conv Layer = 2, Dropout = True, Optimizer = ADAM)

```
CNN : two_conv = True, dropout = True, optimizer = adam
Epoch 1/10 Train Loss: 1.4454 | Train Acc: 52.60% | Val Acc: 85.40%
Epoch 2/10 Train Loss: 0.5607 | Train Acc: 81.98% | Val Acc: 93.00%
Epoch 3/10 Train Loss: 0.3715 | Train Acc: 88.76% | Val Acc: 94.20%
Epoch 4/10 Train Loss: 0.2572 | Train Acc: 92.54% | Val Acc: 95.00%
Epoch 5/10 Train Loss: 0.2207 | Train Acc: 93.52% | Val Acc: 96.40%
Epoch 6/10 Train Loss: 0.1802 | Train Acc: 94.40% | Val Acc: 96.50%
Epoch 7/10 Train Loss: 0.1591 | Train Acc: 95.32% | Val Acc: 97.00%
Epoch 8/10 Train Loss: 0.1307 | Train Acc: 96.00% | Val Acc: 97.30%
Epoch 9/10 Train Loss: 0.1175 | Train Acc: 96.58% | Val Acc: 97.60%
Epoch 10/10 Train Loss: 0.1112 | Train Acc: 96.34% | Val Acc: 97.60%
Test acc : 98.40%
```



2) Dropout = False (BASELINE)



■ dropout이 일반화 성능 향상에 도움이 되었는가?

- 이론적으로는 과적합을 억제함으로써 일반화 성능을 높여주지만 본 실험에서는 데이터가 작고 학습 횟수 또한 짧아서 과적합현상 자체가 일어나지 않아 눈에 띄는 성능 향상이 관찰되지 않았다.

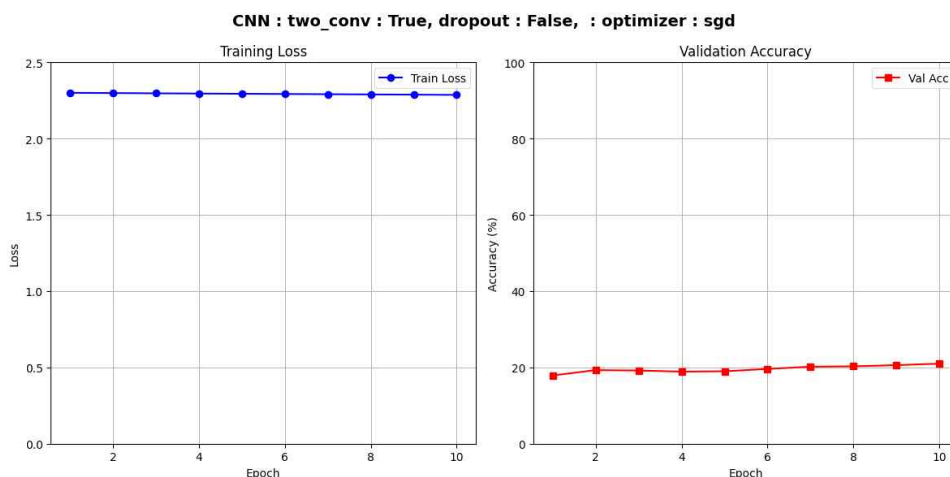
■ train accuracy와 validation accuracy의 차이가 어떻게 변했는가?

- Dropout을 켜면 학습 페널티로 인해 Train Acc가 억제되고 Val Acc와의 격차가 줄어든다.

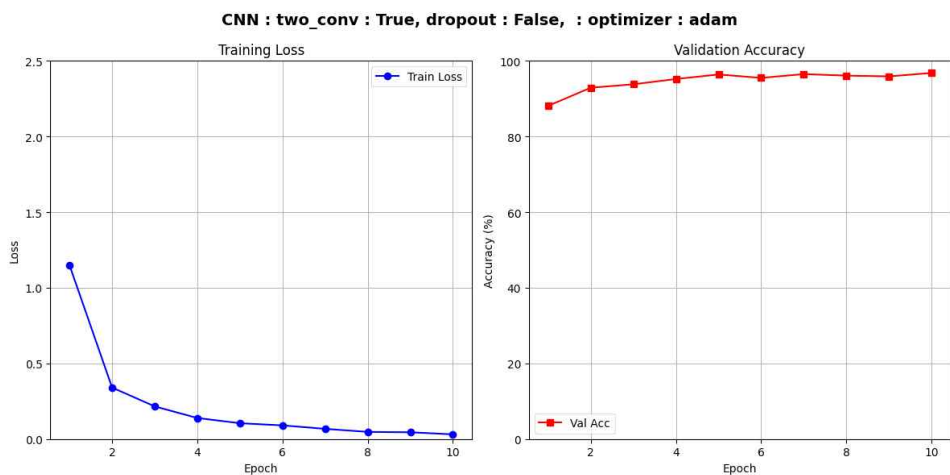
2.5. Optimizer 비교

1) Optimizer = SGD (Conv Layer = 2, Dropout = False, Optimizer = SGD)

```
... CNN : two_conv = True, dropout = False, optimizer = sgd
Epoch 1/10 Train Loss: 2.3003 | Train Acc: 16.28% | Val Acc: 17.90%
Epoch 2/10 Train Loss: 2.2986 | Train Acc: 19.00% | Val Acc: 19.30%
Epoch 3/10 Train Loss: 2.2970 | Train Acc: 19.80% | Val Acc: 19.20%
Epoch 4/10 Train Loss: 2.2955 | Train Acc: 19.28% | Val Acc: 18.90%
Epoch 5/10 Train Loss: 2.2940 | Train Acc: 19.62% | Val Acc: 19.00%
Epoch 6/10 Train Loss: 2.2925 | Train Acc: 19.66% | Val Acc: 19.60%
Epoch 7/10 Train Loss: 2.2911 | Train Acc: 20.14% | Val Acc: 20.20%
Epoch 8/10 Train Loss: 2.2896 | Train Acc: 21.26% | Val Acc: 20.30%
Epoch 9/10 Train Loss: 2.2881 | Train Acc: 21.00% | Val Acc: 20.60%
Epoch 10/10 Train Loss: 2.2865 | Train Acc: 21.62% | Val Acc: 21.00%
Test acc : 21.30%
```



2) Optimizer = ADAM (BASELINE)



■ 어떤 optimizer가 더 빠르게 수렴하였는가?

- Adam Optimizer가 초기 1-2에폭 만에 높은 acc를 달성하며 압도적으로 빠르게 수렴했다.

■ 어떤 optimizer가 더 안정적이었는가?

- Adam이 안정적이었다. 특히 본 실험에서는 learning rate=0.001이었기 때문에 SGD는 더더욱 학습하기 어렵고 Saddle point를 탈출하지 못하는 모습을 보였다.

3. 결론

3.1. HW5 (Python 구현) 과의 차이

1) 구현 난이도 차이

HW5는 순전파 / 역전파 행렬 연산과 미분 공식 등을 모두 직접 수학적으로 유도해야 해서 어렵고 오래 걸렸으나, Pytorch는 제공되는 레이어와 자동 계산 덕분에 빠르게 구현할 수 있었다. 그러나 직접 구현 방식의 직관성에 비해 pytorch의 객체 지향 구조나 낯선 내부 함수들을 이해하는 데에서 오히려 난이도가 높다고 느꼈다.

2) 코드 길이 및 가독성 차이

HW5는 모든 것을 직접 구현했기 때문에 코드 길이가 매우 길었고 Pytorch에서는 nn.Sequential 등을 활용할 수 있어 짧아졌다. 또한 모델 구조가 간결하게 보였다. 그러나 디버깅 등 코드를 직접 분석하며 수정해야 할 때는 직관적인 HW5 코드가 더 가독성이 좋았다.

3) 학습 속도 차이

CPU 기반의 Numpy 연산에 의존했던 HW5와 달리 Pythorch는 cuda gpu 가속을 활용해서 압도적인 학습 속도 향상을 경험했다.

4) 모델 수정의 편의성 차이

HW5는 모든 코드를 수작업으로 고쳐야 했으나 Pytorch는 한두 줄만에 자동으로 모든 연산이 조정되어 편리했다.

5) 직접 구현 방식과 프레임워크 방식의 장단점

직접 구현 방식은 딥러닝의 수학적 작동 원리를 완벽히 이해할 수 있었고, 프레임워크는 생산성과 성능이 압도적이지만 내부 연산을 이해하기에 조금 더 어렵다는 단점이 있었다.

3.2. 결과 분석

1) PyTorch로 구현한 CNN은 HW5의 직접 구현 CNN과 비교할 때 어떤 장점이 있었는가?

복잡한 계산을 자동으로 해 주는 autograd 덕분에 안정적인 모델 구축이 가능했고 GPU 가속 및 다양한 옵티마이저를 쉽게 적용하여 더 높은 성능을 확보할 수 있었다.

2) Conv layer 수가 증가하면 항상 성능이 향상되는가?

MNIST처럼 해상도가 낮고 단순한 데이터셋은 단일 Conv 레이어만으로도 공간적 특징 추출이 가능하기 때문에 레이어를 깊게 쌓는 것이 무의미했다.

3) dropout은 과적합 완화에 효과가 있었는가?

이론적으로는 효과가 크지만 본 실험에서는 데이터 양과 에폭이 적어 모델이 과적합되지 않은 상태였으므로 드롭아웃의 유의미한 성능 차이가 관찰되지 않았다.

4) SGD와 Adam 중 어느 optimizer가 더 적절하였는가?

Adam이 훨씬 적절했다. SGD는 통제된 너무 작은 학습률 때문에 수렴하지 못하고 정체된 반면 Adam은 Acc가 빠르게 수렴했다.

5) PyTorch를 사용할 때 가장 편리하다고 느낀 점은 무엇인가?

모든 가중치의 미분값이 알아서 계산되는 시스템과 옵티마이저를 손쉽게 스위칭할 수 있는 점이 가장 편리했다.

6) 직접 구현 방식이 교육적으로 왜 중요한가?

이론대로 수식을 직접 써가며 구현함으로써 알고리즘의 내부 메커니즘을 빠르게 체득하고 모델에서 발생할 수 있는 다양한 문제를 디버깅하는 데에 도움이 된다.